

SENIOR ENGINEER INTERVIEW PREP

Gokulakonda Gautham

DOCUMENT 2 OF 10

Async, Concurrency & Event Loop

14 Questions | Deep Technical Depth | Event Loop Internals

Async, Concurrency & Event Loop

Q1: What is the event loop and how does it work internally?

The event loop is a scheduler that runs in a single thread. It maintains a queue of ready-to-run coroutines and a set of IO callbacks. It loops forever: pick a ready coroutine, run it until it awaits something, register the IO callback, then move to the next coroutine.

```
# Conceptual model of the event loop
while True:
    # 1. Run all coroutines that are ready
    for task in ready_queue:
        task.step()          # runs until next await

    # 2. Poll OS for IO completions (epoll/kqueue/IOCP)
    events = selector.select(timeout)

    # 3. Wake up coroutines whose IO is done
    for event in events:
        callback = io_waiters[event.fd]
        ready_queue.append(callback)
```

OS primitive: asyncio uses the OS's IO multiplexing: epoll on Linux, kqueue on macOS, IOCP on Windows. A single select()/epoll_wait() call monitors thousands of file descriptors simultaneously.

asyncio.get_event_loop(): Returns the running loop for the current thread. In Python 3.10+, always prefer asyncio.get_running_loop() — it raises if no loop is running rather than creating a new one silently.

uvloop: A drop-in replacement for the default asyncio event loop, written in Cython on top of libuv (the same C library Node.js uses). 2-4x faster. FastAPI/Uvicorn use this in production.

Q2: How does asyncio work internally — from coroutine to IO and back?

Understanding the full chain — from your async function to the OS network call and back — is what separates engineers who use async from engineers who debug it.

```
async def fetch(url):
    async with httpx.AsyncClient() as client:
        response = await client.get(url)  # <-- what happens here?
    return response.text

# Step 1: await suspends the coroutine, returns a Future to the event loop
# Step 2: httpx registers the socket with the event loop selector
# Step 3: event loop calls selector.select() – kernel watches the socket
# Step 4: data arrives – kernel signals the fd
# Step 5: event loop wakes up, calls the registered callback
# Step 6: callback sets the Future result
# Step 7: our coroutine resumes right after the await
```

Future vs Task: A Future is a promise — a container for a result that doesn't exist yet. A Task wraps a coroutine and schedules it on the event loop. asyncio.create_task() is the normal way to schedule concurrent work.

```
# Sequential – total time = sum of all waits
result1 = await fetch('https://api1.com')
result2 = await fetch('https://api2.com')

# Concurrent – total time = max of all waits
result1, result2 = await asyncio.gather(
    fetch('https://api1.com'),
```

```
fetch('https://api2.com'),  
)
```

Q3: Difference between async and parallelism

This is a commonly confused distinction. Async is about concurrency — making progress on multiple tasks by interleaving them on one thread. Parallelism is about simultaneity — multiple tasks literally running at the same instant on multiple cores.

Async (concurrency): Single thread. When one coroutine waits for IO, the event loop runs another. No parallel CPU execution. Total CPU time = sequential CPU time. Benefit: eliminates idle waiting time.

Parallelism: Multiple threads or processes running simultaneously on different CPU cores. True simultaneous computation. Benefit: reduces total CPU time for compute-heavy work.

Can you have both?: Yes. Run multiple Uvicorn worker processes (multiprocessing parallelism), each running their own asyncio event loop (concurrency within each process). This is exactly how production FastAPI is deployed.

```
# uvicorn with multiple workers = parallelism + async concurrency  
uvicorn main:app --workers 4 --loop uvloop  
  
# Each worker is an independent process with its own event loop  
# 4 cores × N concurrent coroutines per core
```

Q4: What happens when await is encountered?

At the bytecode level, await is implemented with a chain of yield-from operations all the way down to a Future. When the innermost object yields, control propagates up to the event loop.

```
# Under the hood, 'await expr' is roughly:  
# 1. Call expr.__await__() to get an iterator  
# 2. yield from that iterator  
# 3. When the iterator raises StopIteration(value), that's the result  
  
# Coroutines ARE generators internally  
# 'async def' creates a coroutine object  
# 'await' is syntactic sugar for 'yield from' on an awaitable  
  
async def demo():  
    x = await some_future    # suspends here, resumes with future's result  
    return x  
  
# The event loop drives this by calling .send(None) to start,  
# then .send(result) to resume after IO completes
```

Awaitable types: Three things are awaitable: coroutines (async def functions), asyncio.Future objects, and asyncio.Task objects. Anything implementing __await__ that returns an iterator.

What does NOT suspend: Pure computation. If your coroutine does 1 million iterations of a for loop with no awaits, it holds the event loop hostage for the entire duration. Other coroutines cannot run during this time.

Q5: Why is async useful for IO-bound tasks but not CPU-bound?

The answer lies in where time is spent. IO-bound tasks spend most of their time waiting for external responses. CPU-bound tasks spend all their time executing instructions.

IO-bound: Making a DB query: 1ms to send, 50ms waiting, 1ms to receive. That 50ms of idle is wasted if synchronous. Async fills it with other work — effectively multiplying throughput by 50x without adding hardware.

CPU-bound: Computing a hash: 100ms of pure CPU. There is no idle waiting. Async cannot help — the event loop cannot switch to other coroutines because this one never awaits. It's worse than sync because async has overhead.

```
# IO-bound: async wins
async def handle_1000_requests():
    tasks = [fetch(url) for url in urls]
    return await asyncio.gather(*tasks) # all in-flight simultaneously

# CPU-bound: async HURTS – blocks the loop
async def bad_idea():
    result = compute_primes(10_000_000) # freezes all other coroutines

# CPU-bound correct approach: offload to thread or process pool
async def good_approach():
    loop = asyncio.get_running_loop()
    result = await loop.run_in_executor(None, compute_primes, 10_000_000)
```

Q6: What are coroutines and how are they different from functions?

A regular function runs to completion when called. A coroutine is a function that can suspend its execution mid-way, yield control, and later resume exactly where it left off — retaining all its local variables.

```
# Regular function – runs to completion
def add(a, b):
    return a + b

# Coroutine – calling it returns a coroutine object, does NOT run it
async def fetch_user(user_id):
    data = await db.get(user_id) # suspends here
    return data

coro = fetch_user(42) # nothing runs yet
result = await coro # NOW it runs (must be inside async context)
```

State preservation: When a coroutine suspends at await, its entire stack frame — local variables, execution pointer, exception state — is preserved. This is what distinguishes coroutines from regular function calls.

Coroutine lifecycle: Created → Scheduled (create_task) → Running → Suspended (at await) → Running → Done/Cancelled. A coroutine that is created but never awaited is garbage-collected and Python warns you: 'coroutine was never awaited'.

async for / async with: Coroutines extend to iteration and context management. async for works with objects implementing __aiter__ and __anext__. async with works with __aenter__ and __aexit__. Streaming DB results with async for is a common production pattern.

Q7: How does task scheduling work in asyncio?

The event loop maintains a deque of ready callbacks. Tasks are scheduled by appending to this deque. The loop processes them in FIFO order, running each until it yields.

```
import asyncio
```

```

async def task_a():
    print('A start')
    await asyncio.sleep(0)    # yields to event loop
    print('A end')

async def task_b():
    print('B start')
    await asyncio.sleep(0)
    print('B end')

async def main():
    # create_task schedules both immediately
    t1 = asyncio.create_task(task_a())
    t2 = asyncio.create_task(task_b())
    await t1
    await t2

# Output: A start → B start → A end → B end
# Interleaved because sleep(0) yields control

```

asyncio.sleep(0): A zero-duration sleep is a cooperative yield — it suspends the current coroutine and lets the event loop run other pending tasks. Useful in long CPU loops to avoid starving other coroutines.

asyncio.gather vs asyncio.wait: gather() runs coroutines concurrently and returns all results in order. wait() gives you more control — you can react as tasks complete (FIRST_COMPLETED) or handle partial failures independently.

Q8: Coroutine vs thread — when to choose which?

This is an architectural decision that affects scalability, debuggability, and correctness. Neither is universally better.

Memory overhead: A thread typically uses 1-8MB of stack space. A coroutine uses only what its local variables require — often a few KB. You can run 100,000 coroutines; you cannot run 100,000 threads on most systems.

Switching cost: Thread context switches are OS-level — save/restore registers, TLB flush — typically microseconds. Coroutine switches are just Python function calls — nanoseconds.

Shared state: Coroutines share state safely without locks (single thread, cooperative scheduling — you know exactly when context switches happen). Threads require explicit locking everywhere.

Debugging: Async stack traces can be harder to read. Threads have predictable call stacks. asyncio.Task has get_stack() for debugging suspended coroutines.

```

# Choose coroutines for: high-concurrency APIs, websockets, streaming
# Choose threads for: blocking legacy libs, CPU-light background jobs

# Mixing them correctly:
async def hybrid():
    # Run blocking library in thread without blocking the event loop
    result = await asyncio.to_thread(blocking_library_call, arg)
    # Now back to async for IO
    await async_io_call(result)

```

Q9: What causes blocking in async systems and how do you detect it?

Blocking is the #1 production issue with async systems. A single blocking call can freeze your entire server, causing timeout cascades across all concurrent requests.

Common blocking culprits: `time.sleep()` instead of `asyncio.sleep()`. `requests` library instead of `httpx/aiohttp`. Synchronous ORM calls (SQLAlchemy without `async`). CPU-heavy computation without offloading. `os.path` operations on network mounts.

```
# WRONG – blocks the entire event loop
async def get_user(user_id):
    time.sleep(1)                # freezes loop
    result = requests.get('http://...') # blocks thread
    return db.execute('SELECT ...')    # sync ORM blocks

# CORRECT
async def get_user(user_id):
    await asyncio.sleep(1)        # yields to loop
    async with httpx.AsyncClient() as c: # non-blocking
        result = await c.get('http://...')
    return await db.fetch_one('SELECT ...') # async ORM
```

Detection: Use `asyncio`'s debug mode: `PYTHONASYNCIODEBUG=1`. It logs coroutines that run longer than 100ms without yielding. Or install `aiomonitor` for live introspection of running tasks.

The thundering herd problem: If a shared resource (DB connection pool) is exhausted, all coroutines waiting for a connection pile up. Use `asyncio.Semaphore` to cap concurrency on expensive resources.

Q10: What happens if you call a blocking DB call inside an async API?

It freezes the event loop. Every other concurrent request stops making progress until the blocking call returns. Under load, this compounds — requests queue up, timeouts fire, the server appears hung.

```
# Scenario: FastAPI endpoint with SQLAlchemy sync session
@app.get('/user/{id}')
async def get_user(id: int, db: Session = Depends(get_db)):
    # db.query() is SYNCHRONOUS – blocks the event loop
    user = db.query(User).filter(User.id == id).first()
    return user

# Under 100 concurrent requests: only 1 runs at a time during the DB call
# Effective concurrency = 1 during query execution

# Fix option 1: Use async SQLAlchemy
from sqlalchemy.ext.asyncio import AsyncSession
async def get_user(id: int, db: AsyncSession = Depends(get_db)):
    result = await db.execute(select(User).where(User.id == id))
    return result.scalar_one()

# Fix option 2: run_in_executor (for legacy code)
async def get_user(id: int):
    user = await asyncio.to_thread(sync_db_query, id)
    return user
```

Connection pool sizing: With async DB (`asyncpg`, `aiopg`), you need far fewer connections than with sync. 10 async connections can handle 10,000 concurrent queries. With sync, you'd need 10,000 connections — which would kill your DB.

Q11: How does FastAPI support async? ASGI vs WSGI?

FastAPI is built on Starlette which is an ASGI framework. ASGI (Asynchronous Server Gateway Interface) is the spiritual successor to WSGI, designed for async Python.

WSGI: The old standard (Django, Flask). Synchronous. Each request gets a thread. The server calls `app(environ, start_response)`. No native support for WebSockets, streaming, or long-polling.

ASGI: The new standard (FastAPI, Django 3.0+ channels, Litestar). Async-first. The server calls `await app(scope, receive, send)`. Scope describes the connection; receive is an async callable for incoming data; send is for responses.

```
# Minimal ASGI app – what FastAPI builds on top of
async def app(scope, receive, send):
    assert scope['type'] == 'http'
    await send({
        'type': 'http.response.start',
        'status': 200,
        'headers': [[b'content-type', b'application/json']],
    })
    await send({'type': 'http.response.body', 'body': b'{"ok": true}'})

# FastAPI wraps this: route matching, dependency injection,
# Pydantic validation, OpenAPI spec generation – all on top of ASGI
```

Async vs sync routes in FastAPI: `async def` routes run directly on the event loop. Regular `def` routes are automatically run in a thread pool via `run_in_executor` — so they don't block the event loop, but they also can't use `await`. Use `async def` by default; use `def` only for CPU-bound or when wrapping blocking code intentionally.

Q12: `asyncio.gather` vs `asyncio.wait` vs `TaskGroup` — what's the difference?

All three run coroutines concurrently, but they differ in error handling, cancellation semantics, and ergonomics.

`asyncio.gather()`: Simplest API. Returns results in order matching inputs. If any task raises and `return_exceptions=False` (default), `gather` cancels all others and re-raises. Use for fixed-count concurrent calls.

`asyncio.wait()`: More control. Returns (done, pending) sets. Can use `FIRST_COMPLETED` to react as each task finishes. Good for competitive racing or partial failure handling.

`TaskGroup (Python 3.11+)`: Structured concurrency. Tasks created with `tg.create_task()` inside 'async with `asyncio.TaskGroup()` as `tg`'. If any task fails, all others are cancelled. More structured than `gather`, prevents orphaned tasks.

```
# gather – concurrent, ordered results
results = await asyncio.gather(coro1(), coro2(), coro3())

# wait – react to first completion
tasks = {asyncio.create_task(c) for c in [coro1(), coro2()]}
done, pending = await asyncio.wait(tasks, return_when=asyncio.FIRST_COMPLETED)

# TaskGroup – Python 3.11+ structured concurrency
async with asyncio.TaskGroup() as tg:
    task1 = tg.create_task(coro1())
    task2 = tg.create_task(coro2())
# Both tasks guaranteed complete (or all cancelled) here
result1 = task1.result()
```

Q13: How do you handle cancellation in async code?

Task cancellation is cooperative in `asyncio`. Cancelling a task injects a `CancelledError` at the next `await` point. Ignoring or swallowing `CancelledError` is a critical bug that prevents clean shutdown.

```
import asyncio

async def worker():
```

```

try:
    while True:
        await asyncio.sleep(1)    # CancelledError raised here on cancel
        await do_work()
    except asyncio.CancelledError:
        print('Cleaning up resources before exit')
        await cleanup()           # OK to await in cancellation handler
        raise                     # CRITICAL: must re-raise CancelledError

task = asyncio.create_task(worker())
await asyncio.sleep(5)
task.cancel()                    # injects CancelledError at next await
await task                      # wait for clean cancellation

```

Timeout handling: `asyncio.timeout()` (3.11+) or `asyncio.wait_for()` — both cancel the inner task on timeout. Always handle `CancelledError` in long-running tasks to release resources (DB connections, file handles, locks).

Q14: What is `asyncio.Semaphore` and when do you use it?

A Semaphore limits concurrency — it's a counter that prevents more than N coroutines from executing a section simultaneously. Essential for rate limiting, connection pool protection, and external API throttling.

```

import asyncio, httpx

sem = asyncio.Semaphore(10)    # max 10 concurrent requests

async def fetch(client, url):
    async with sem:             # blocks if 10 already running
        response = await client.get(url)
        return response.json()

async def fetch_all(urls):
    async with httpx.AsyncClient() as client:
        tasks = [fetch(client, url) for url in urls]
        return await asyncio.gather(*tasks) # 1000 tasks, max 10 at once

# Without semaphore: 1000 simultaneous connections → overwhelm server
# With semaphore: exactly 10 active, rest queued — polite and safe

```

asyncio.Lock: Mutual exclusion — only 1 coroutine in the critical section. Use for protecting shared mutable state (in-memory cache, counter) in async code. Unlike `threading.Lock`, this is NOT blocking — it yields to the event loop while waiting.